PROJECT AND TEAM INFORMATION

Project Title

Fake Emergency Call Detection and Priority Routing System

Student / Team Information

Team Name: *Team ID- DS-CPP-2026-T010*	*Novexa*
Team member 1 (Team Lead) *(Last Name, name: student ID: email, picture):*	*Avika, Dixit -- 2510018349* *avika51110@gmail.com* 
Team member 2 *(Last Name, name: student ID: email, picture):*	*Bisht, Harshita -- 2510360038* *bishtharshi25@gmail.com* 
Team member 3 *(Last Name, name: student ID: email, picture):*	*Mittal, Saksham -- 2510014103* *sakshammittal.geu@gmail.com* 
Team member 4 *(Last Name, name: student ID: email, picture):*	*Bhandari, Nitin Singh -- 2512510026* *nitinsinghbhandari9927@gmail.com* 

PROPOSAL DESCRIPTION

Motivation

When an emergency occurs, police, fire, and ambulance dispatchers often receive a high volume of emergency calls that report the same incident, but give slightly different details. Processing each call individually wastes time and resources responding to the same event. This practice does not serve the best interests of those calling in an emergency. In addition, callers may provide inaccurate and even conflicting information about the emergency.

My Smart Emergency Call Verification and Priority Routing project proposes to address these issues by developing an efficient method to organize and prioritize emergency calls by grouping them into a single event to ensure faster and more efficient emergency response. This project is a great way to demonstrate my knowledge of Data Structures and C++ Object-Oriented Programming concepts and implement them to solve real-life problems.

I will use Hash Maps to store and retrieve data about emergency incidents and their details, Queues to store the emergency calls, and Priority Queues to prioritize emergency incidents based on their importance and urgency. The project will be helpful in assisting emergency response dispatchers in making decisions faster when responding to emergency calls.

State of the Art / Current solution

In the systems we have in place right now, trained human operators collect the emergency calls and manually document all details. They inquire the location of the emergency, the type, the state of the emergency, and others. Once all the information is collected, a dispatcher has to analyze the whole situation and determine what resources are needed.

But the big problem with such a system is that there are cases where one emergency can generate tens of emergency calls (especially if it happens at a crossroad). In this case, without some advanced tools, an operator may think that ten different accidents have occurred, instead of one.

The same problem happens when people describe the emergency in a wrong or mixed way due to stress. Large-scale governmental programs use heavily overloaded databases and GPS systems for processing such information. However, our project implements a light-weighted solution based on a basic set of Data Structures and Algorithms that allow automatically identifying such emergency reports and grouping them together before a human operator analyzes the situation.

Project Goals and Milestones

A Prototype which is able to process emergency calls,find related reports,create consolidation incidents and prioritize them for dispatch is the principle objective of the current project.

GOALS-:

1.Create a command line interface for receiving and storing emergency calls.
2.Use aa Quene for managing incoming calls.
3.Use Hash map for identification of duplicate calls.
4.Use Graph for grouping related emergency report
5.Create a Call analyzer highlighting incomplete,duplicate,suspicious and conflicting information.
6.Calculate the priority if an incident based on its severity and amount of available information.
7.Use Priority Quene fo incident sorting.
8.Create dispatcher module for allocating proper response team.
9.Create a simple interface for monitoring calls,incident and dispatch

MILESTONES-:

1.Requirement analysis and system design.
2.Design OOPs classes and basic data structure.
3.Implement Quene and Hash map functions.
4.Implement Graph based on incident grouping.
5.Calculation of priority and implement Priority Quene.
6.Connect all separate parts to main functions.
7.Test with duplicate,incomplete and suspecious emergency report.
8.Final implementation of whole project

Project Approach

Project is going to be implemented as a modular C++ application by applying concepts of Object-Oriented Programming and Data Structures. Application is going to be processing emergency calls in the pipeline manner, when each call is going to be put first in the incoming queue. All critical information like caller's information, emergency location, type of emergency, description, urgency and confidence are going to be kept.

CallAnalyzer is going to be the specialized class responsible for processing of each call and detection of its duplication via Hash Map structure and related information fields. Duplicates and alike reports are going to form the graph and DSU is going to be used for merging separate reports into one incident. New Incident object is going to be generated based on the information contained in the emergency call. Special metric is going to be calculated for each incident and put in the priority queue for sorting by its importance. The metric is going to consider the urgency of incident, number of affected people and amount of information available for the incident.

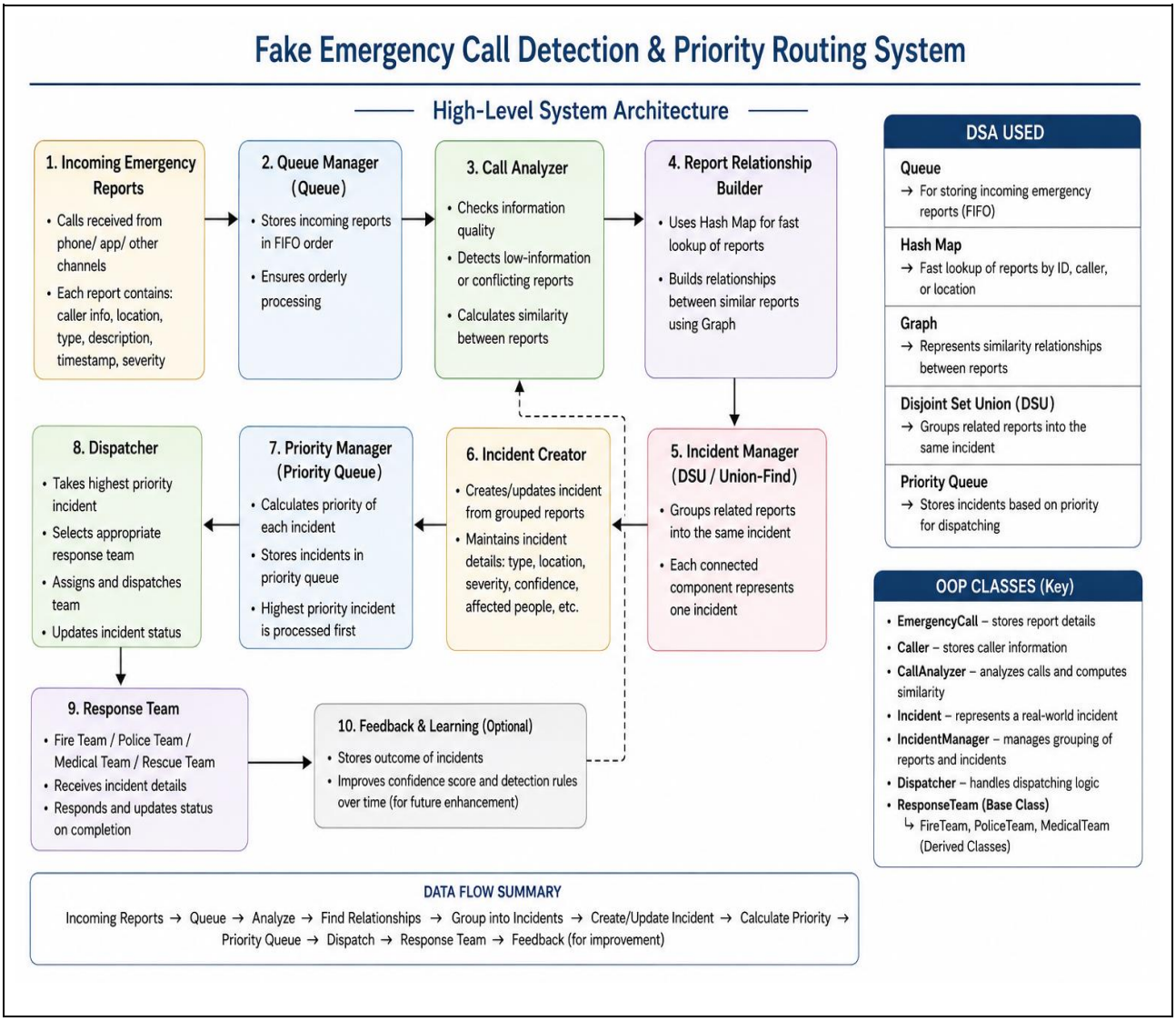
Following OOP classes are going to be used for implementation of the project:

EmergencyCall
Caller
Incident
CallAnalyzer
Dispatcher
ResponseTeam

Application is able to be compiled in the console application for the time being with necessary commands. Code is going to be implemented in C++ with using of STL/data-structure implementations, OOP concepts (encapsulation, polymorphism, inheritance, constructors, destructors, etc.) and necessary functions.

Different kinds of tests are going to be used for testing of the application, like duplicate calls, nearby incidents, low information emergency reports and other edge cases.

System Architecture (High Level Diagram)



Project Outcome / Deliverables

The final result will be a working prototype of the system that is able to implement a Smart Emergency Call Verification, Incident Clustering and Priority Routing in C++.

The primary deliverables will comprise such modules as

1. The module for entering and storing emergency calls;
2. The Queue-based incoming call manager;
3. The Hash Map-based duplicate detector;
4. The Graph/Disjoint Set Union-based incident grouper;
5. The Call Analyzer managing low-information, duplicate, suspicious and conflicting reports;
6. The Incident Manager creating and updating unified incidents;
7. The Priority Queue, which will manage the incidents according to their severity and priority;
8. The Dispatcher, which will assign the appropriate response teams;
9. The console-based UI, which will demonstrate the whole workflow; and
10. The test cases and documentation, which will detail the DSA and OOP concepts implemented.

The project will showcase how the basic concepts of DSA and OOP could be combined to address the real-life emergency-response scenario.

Assumptions

• The system works under the assumption that emergency calls contain essential information such as location, emergency type, description, and severity
• The location data is assumed to be available in a simplified manner rather than relying on real-time GPS data.
• Reports that occur within an appropriate time and location threshold are likely to be connected
• Duplicate and similarity identification will be dependent on specific rules and data accessible to the system.
• The system will perform report categorization for verification and prioritization but will not make independent verification if a caller is lying or making unreasonable emergency decisions.
• The verification and ultimate dispatch decision will be dependent on a human dispatcher.

References

1. Bjarne Stroustrup, The C++ Programming Language , Addison Wesley.
2. Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein, Introduction to Algorithms , MIT Press.
3. GeeksforGeeks – Data Structures and Algorithms , C++ and Graph/DSU concepts.
4. cppreference — C++ language and Standard Library reference.
5. Materials related to the course lectures and laboratory of the Data Structures and Object-Oriented Programming .